Reviewer Pack — Minimal Order of Representations over p-Adic Fields and Comm…

Entry 6a047b720ec4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 12:48:23 UTC

Minimal Order of Representations over p-Adic Fields and Communication Complexity Rank Variance

Entry ID: `6a047b720ec4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 12:48:23 UTC

1. Conjecture

Field A (mathematical branch): p-adic Number Theory **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

Statement:

> For all boolean functions f with n variables, the minimal order of a non-degenerate representation of the associated vector space over the p-adic field Q_p is polynomially related to the variance in the ranks of the communication complexity matrices for f. Specifically, let σ_min(f) be the minimal order and C(f) be the matrix representing communication complexity. Then, σ_min(f) = Θ(poly(n) * Var(Rank(C(f))))

Rationale (proposer's reasoning):

> p-adic fields offer a non-Archimedean setting that can potentially capture the algebraic structure of boolean functions in a way that is relevant to their complexity. The minimal order of representations could provide insight into the inherent difficulty of computing f, which in turn might be reflected in the rank variance of its communication complexity matrix.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0674505b3ef7739f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, across all boolean functions f with n variables, the ratio σ_min(f)/Var(Rank(C(f))) is polynomially bounded by n.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	SAFE	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=-9, elapsed=108.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def min_non_degenerate_representation(f):
        n = int(math.log2(len(f)))
        V = [f[i] for i in range(2**n) if f[i] != 0]
        return len(V)

    def communication_complexity_matrix(f):
        n = int(math.log2(len(f)))
        C = [[0] * (2**n) for _ in range(2**n)]
        for x in range(2**n):
            for y in range(2**n):
                if f[x] == f[y]:
                    C[x][y] = 1
        return C

    def matrix_rank(matrix):
        m, n = len(matrix), len(matrix[0])
        rank = 0
        for i in range(m):
            if any(matrix[i][j] != 0 for j in range(n)):
                rank += 1
                for j in range(i+1, m):
                    factor = matrix[j][i] / matrix[i][i]
                    for k in range(n):
                        matrix[j][k] -= factor * matrix[i][k]
        return rank

    def variance(lst):
        mean = sum(lst) / len(lst)
```

```
        return sum((x - mean) ** 2 for x in lst) / len(lst)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_boolean_function(n)
        sigma_min = min_non_degenerate_representation(f)
        C = communication_complexity_matrix(f)
        rank_C = matrix_rank(C)
        results.append((n, sigma_min, rank_C))

    if not results:
        return {
            "metric_name": "sigma_min / Var(Rank(C))",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "empty_results"
        }

    sigma_mins = [r[1] for r in results]
    rank_Cs = [r[2] for r in results]
    var_rank_C = variance(rank_Cs)

    if var_rank_C == 0:
        return {
            "metric_name": "sigma_min / Var(Rank(C))",
            "metric_value": None,
            "instances_tested": len(results),
            "n_max": max(r[0] for r in results),
            "conjecture_holds": False,
            "counterexample": "variance_zero"
        }

    ratio = sum(sigma_mins) / var_rank_C
    return {
        "metric_name": "sigma_min / Var(Rank(C))",
        "metric_value": ratio,
        "instances_tested": len(results),
        "n_max": max(r[0] for r in results),
        "conjecture_holds": True if ratio <= 1 else False,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)
```

```
if all(r["conjecture_holds"] for r in results):
    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    std_ratio = math.sqrt(sum((r["metric_value"] - mean_ratio) ** 2 for r in results) / len(re
    support_fraction = 1.0
else:
    first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
    mean_ratio = sum(r["metric_value"] for r in results[:first_failing_seed]) / first_failing_
    std_ratio = math.sqrt(sum((r["metric_value"] - mean_ratio) ** 2 for r in results[:first_fa
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

print(f"RESULT: {'SUPPORTED' if all(r['conjecture_holds'] for r in results) else 'FALSIFIED'}
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
(empty)
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test with a different boolean function or under different conditions to ensure it completes successfully and provides the necessary data for verification.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13509
2	propose	ollama_remote	glm4:latest	0	0	16282
3	preregistration	ollama_remote	glm4:latest	0	0	15128
4	novelty	ollama_remote	glm4:latest	0	0	8544
5	novelty	ollama_remote	glm4:latest	0	0	8550
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	33068
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	52608
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15081
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14384
10	judge	ollama_remote	glm4:latest	0	0	104831

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 281985 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/6a047b720ec4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6a047b720ec4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/6a047b720ec4.tar.gz` (if generated)